

Arduino Cookbook : De la Théorie à la Pratique

Chapitre 11 : Affichages Avancés (Displays)

Présenté par : Dr B. DIOP

29 mars 2026

Référence : Michael Margolis, *Arduino Cookbook*, 2nd Edition, O'Reilly

11.0 Introduction : Au-delà des simples LEDs

Communiquer des données complexes

Si les LEDs suffisent pour des états binaires (Allumé/Éteint), les projets avancés nécessitent d'afficher des valeurs mesurées, des menus, ou des graphiques pour l'utilisateur.

Dans ce chapitre, nous allons explorer :

- Le formatage avancé du texte sur les écrans LCD standards (HD44780).
- La création de caractères personnalisés (Custom Characters).
- Les écrans graphiques monochromes (GLCD).
- Les écrans couleurs TFT (Thin-Film Transistor).
- La technologie OLED pour un contraste infini.

11.1 Rappel : L'écran LCD Textuel

L'écran LCD 16x2 ou 20x4 (vu au chapitre 7) reste l'affichage le plus robuste et le plus utilisé dans l'industrie pour les interfaces homme-machine simples.

Gestion de l'espace :

- Un écran 16x2 possède 32 "cases" en tout.
- Il n'y a pas de retour à la ligne automatique ! Si vous écrivez 17 caractères sur la ligne 1, le 17ème n'apparaîtra pas au début de la ligne 2, mais sera caché en mémoire.
- Il faut utiliser `lcd.setCursor(colonne, ligne)` de manière stricte.

11.2 Formater l'affichage des nombres

Problème fréquent : Lorsqu'on affiche une valeur qui passe de "100" à "99", le dernier zéro reste à l'écran, affichant "990" !

Solution : Toujours effacer l'ancienne valeur ou utiliser la fonction `sprintf()` pour formater la chaîne avec des espaces constants.

```
#include <LiquidCrystal.h>
LiquidCrystal lcd(12, 11, 5, 4, 3, 2);

void loop() {
    int capteur = analogRead(A0);
    char buffer[16]; // Tableau pour stocker la chaîne formatee

    // %4d signifie : entier decimal occupant 4 espaces (ex: " 99", "1023")
    sprintf(buffer, "Valeur : %4d", capteur);

    lcd.setCursor(0, 0);
    lcd.print(buffer); // Affiche proprement sans laisser de "fantomes"
    delay(200);
}
```

11.3 Créer des caractères personnalisés (Custom Chars)

L'écran HD44780 possède une mémoire ROM contenant l'alphabet, mais aussi une mémoire RAM (CGRAM) permettant de stocker **8 caractères personnalisés**.

- Chaque case de l'écran LCD est une petite matrice de **5 colonnes par 8 lignes** de pixels.
- On dessine le caractère en binaire : 1 allume le pixel, 0 l'éteint.
- Idéal pour afficher des symboles comme "°C", une jauge de batterie, ou une petite animation (Pac-Man).

[Insérer l'image d'une grille 5x8 pixels montrant comment dessiner un smiley]

Code : Afficher un "Cœur" personnalisé

```
#include <LiquidCrystal.h>
LiquidCrystal lcd(12, 11, 5, 4, 3, 2);

// Dessin du coeur pixel par pixel (5x8)
byte coeur[8] = {
    0b00000,
    0b01010,
    0b11111,
    0b11111,
    0b01110,
    0b00100,
    0b00000,
    0b00000
};

void setup() {
    lcd.createChar(0, coeur); // Stocke le motif dans la case memoire 0
    lcd.begin(16, 2);

    lcd.print("I ");
    lcd.write(byte(0)); // Affiche le caractere personnalise
    lcd.print(" Arduino");
}
```

11.4 Les Écrans Graphiques Monochromes (GLCD)

Différence majeure : Contrairement à l'écran LCD textuel qui divise l'écran en "cases" de lettres, le GLCD (Graphic LCD, ex : 128x64 pixels) permet d'allumer et d'éteindre **chaque pixel individuellement**.

- **Le système de coordonnées :** L'origine ($x = 0, y = 0$) se trouve toujours dans le **coin supérieur gauche**.
- L'axe X va vers la droite, l'axe Y va vers le **bas**.
- Nécessite beaucoup de broches (souvent 13 fils) et l'utilisation de la bibliothèque openGLCD ou U8g2.

Code : Dessiner des formes sur un GLCD

Voici comment dessiner des éléments graphiques primitifs en utilisant une librairie graphique standard (comme U8g2).

```
#include <U8g2lib.h>

// Initialisation (depend du modele exact de l'ecran et du cablage)
U8G2_ST7920_128X64_1_SW_SPI u8g2(U8G2_R0, 13, 11, 10, 8);

void setup() {
    u8g2.begin();
}

void loop() {
    u8g2.firstPage();
    do {
        u8g2.drawFrame(0, 0, 128, 64); // Dessine un rectangle (cadre de l'ecran)
        u8g2.drawCircle(64, 32, 20);   // Dessine un cercle au centre (rayon 20)
        u8g2.drawLine(0, 0, 127, 63); // Diagonale
    } while ( u8g2.nextPage() );
    delay(1000);
}
```

11.5 Les Écrans Couleurs TFT (SPI)

Technologie TFT (Thin-Film Transistor) : Permet d'avoir des écrans couleurs (souvent 16-bits, soit 65 536 couleurs) comparables à ceux des anciens téléphones portables.

Le protocole SPI : Puisqu'il y a énormément de pixels à mettre à jour, ces écrans utilisent le bus matériel SPI (Serial Peripheral Interface) très rapide.

- **SCK (Horloge)** : Broche 13 (sur Uno)
- **MOSI (Données)** : Broche 11
- **CS (Chip Select), DC (Data/Command), RST** : Broches numériques au choix.

[Insérer l'image d'un module TFT 1.8" SPI ou ILI9341]

Code : Afficher du texte en couleur sur TFT

La bibliothèque de référence est souvent TFT.h (incluse dans l'IDE) ou Adafruit_ILI9341.

```
#include <TFT.h>
#include <SPI.h>

#define cs    10
#define dc    9
#define rst   8
TFT ecranTFT = TFT(cs, dc, rst);

void setup() {
  ecranTFT.begin();
  ecranTFT.background(0, 0, 0); // Fond noir (R, G, B)

  ecranTFT.stroke(255, 255, 255); // Couleur du texte : Blanc
  ecranTFT.setTextSize(2);        // Taille du texte
  ecranTFT.text("Bonjour !", 10, 20); // (Texte, coord X, coord Y)
}

void loop() {}
```

11.6 Afficher des Images (Bitmaps) sur TFT

Le problème de la mémoire : Une petite image couleur de 128×160 pixels pèse plus de 40 Ko. La mémoire RAM de l'Arduino Uno n'est que de 2 Ko !

La solution :

1. On stocke l'image (au format BMP 24-bits) sur une carte MicroSD.
2. La plupart des écrans TFT possèdent un lecteur de carte SD intégré au dos.
3. L'Arduino lit le fichier morceau par morceau (ligne par ligne) et l'envoie immédiatement à l'écran via le bus SPI, sans jamais stocker l'image complète en RAM.

11.7 Les Écrans OLED (Organic LED)

L'état de l'art pour les petits affichages : Contrairement au LCD qui a besoin d'un rétroéclairage, dans un écran OLED, **chaque pixel produit sa propre lumière.**

Avantages majeurs :

- Contraste infini (les pixels noirs sont réellement éteints).
- Consommation d'énergie très faible.
- Angles de vision parfaits.
- Communication extrêmement simple via **I2C (seulement 4 fils au total : VCC, GND, SDA, SCL).**

11.8 Le module OLED SSD1306 (I2C)

C'est l'écran le plus populaire aujourd'hui pour les projets de taille réduite (0.96 pouces).
Résolution typique : 128x64 pixels monochromes.

Câblage I2C sur Arduino Uno :

- **SDA (Data)** : Broche A4 (ou broche SDA dédiée près de l'USB).
- **SCL (Clock)** : Broche A5 (ou broche SCL dédiée).

[Insérer le schéma de câblage I2C de l'OLED SSD1306]

Code : Utiliser l'écran OLED avec Adafruit

La combinaison des bibliothèques Adafruit_GFX et Adafruit_SSD1306 est le standard industriel.

```
#include <Wire.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>

#define SCREEN_WIDTH 128
#define SCREEN_HEIGHT 64
Adafruit_SSD1306 display(SCREEN_WIDTH, SCREEN_HEIGHT, &Wire, -1);

void setup() {
  // Initialisation a l'adresse I2C 0x3C
  display.begin(SSD1306_SWITCHCAPVCC, 0x3C);

  display.clearDisplay(); // Efface le buffer
  display.setTextSize(1);
  display.setTextColor(SSD1306_WHITE);
  display.setCursor(0, 0);
  display.println(F("Hello OLED!")); // 'F' economise la RAM

  display.display(); // Obligatoire pour envoyer le buffer a l'ecran !
}
```

11.9 Optimisation de la mémoire (La macro F())

Une astuce vitale pour les écrans : Chaque lettre que vous écrivez dans `lcd.print("Mon Texte");` consomme 1 octet de RAM (qui est très limitée).

La Macro F()

En enveloppant vos textes dans `F()`, comme ceci : `Serial.println(F("Texte"));`, vous forcez l'Arduino à lire la chaîne de caractères directement depuis la mémoire Flash (mémoire du programme, très large) au lieu de la copier en RAM. Cela sauve les projets contenant beaucoup de menus !

Résumé du Chapitre 11

- **Textuel vs Graphique** : Utilisez les écrans 16x2/20x4 pour la fiabilité et la simplicité. Passez aux TFT/OLED uniquement si vous avez besoin de graphiques ou de polices variées.
- **Le Buffer (OLED/GLCD)** : N'oubliez pas que sur la plupart des écrans graphiques, les commandes de dessin se font en mémoire. Il faut appeler une fonction comme `display()` pour que les changements apparaissent réellement.
- **Protocole I2C** : L'OLED (4 fils) est beaucoup plus facile à câbler physiquement que les anciens GLCD (13 fils) ou même que le SPI.
- **Gestion de la RAM** : Utilisez toujours la macro `F()` pour les textes fixes lorsque vous développez des interfaces homme-machine complexes.

Fin du Chapitre 11. Souhaitez-vous que nous abordions le Chapitre 12, consacré à la gestion du Temps et des Dates (Using Time and Dates) ?